AWS ACADEMY CLOUD ARCHITECTING · MODULE 6 CHALLENGE LAB

Migrating an On-Premises MariaDB Database to Amazon RDS

Decoupling the monolithic web and database tiers of the Frank & Martha Café

TEAM MEMBER 1

Ishita Goel

Reg No: 24BCI0004

TEAM MEMBER 2

Srijita Bhuniye

Reg No: 24BDS0368

Lab: AWS Academy Cloud Architecting – Challenge (Café) Lab: Migrating a Database to Amazon RDS

The Monolithic Bottleneck at the Café

The Frank & Martha Café runs its web storefront and its database on one EC2 instance (CafeServer). The database holds orders the café relies on for accounting and for planning how many desserts to bake, but nobody has time to maintain it properly.

Resource Contention

Apache/PHP web traffic and MariaDB queries compete for the same CPU and RAM on CafeServer, so a busy period slows both orders and records.

Single Point of Failure

An EC2 crash or OS reboot takes down the storefront and the persistent order records together, with no standby copy.

Heavy Maintenance

Backups depend on manual scripts, the OS and database need regular security patching, and disk capacity must be planned by hand.

PROBLEMS FACED

Pain Points from the Café Stakeholders

The existing design worked at small scale, but it created operational risk and recurring manual work for a team with no database administrator.

Business & staff concerns

- Frank and Martha rely on order data for accounting and for deciding how many desserts to bake, so losing it is unacceptable
- Sofia must keep the database upgraded and patched but rarely has time
- Administering a database is a specialist skill that the café does not want to train for
- Backups are not taken as often as they should be
- Martha wants to cut the labor cost of managing the database

Technical consequences

- One server doing two jobs: web and database workloads slow each other down
- One failure (EC2 crash, reboot, full disk) stops all orders and loses access to records
- Credentials and endpoint tied to the local machine
- No automatic recovery path or point-in-time restore
- Growth means resizing the whole server, with downtime

THEORY

Why a Managed Database? Amazon RDS & the Shared Responsibility Model

Amazon RDS is a managed relational database service. It removes undifferentiated administration but still leaves design, access control and data in the customer’s hands.

AWS manages with RDS

- Hardware, host OS and database engine patching
- Automated backups and point-in-time recovery
- Storage provisioning and scaling
- Optional Multi-AZ failover and monitoring integration
- Physical security and networking infrastructure

The café still manages

- Choosing the engine and instance size (MariaDB here)
- Security groups and network placement
- Database users, passwords and schema design
- Data migration and application connection settings
- Backup retention and who may access what

PROJECT SCOPE

Objectives & Success Criteria

1. Managed Database Provisioning

Deploy MariaDB on Amazon RDS (CafeDatabase) inside the Lab VPC in a private subnet in us-east-1a, with public access disabled.

2. Least-Privilege Network Hardening

Configure the dbSG security group to allow inbound traffic only from CafeSG, so the database (port 3306) is never reachable from the public internet.

3. Zero-Loss Migration & Decoupling

Export the cafe_db data, import it into RDS, update the /cafe/db* secrets in Secrets Manager, and stop the local MariaDB service.

Monolithic EC2 vs. Decoupled Amazon RDS

Pillar	Before: Monolithic EC2	After: Decoupled RDS
Placement	Apache and MariaDB share one EC2 instance	RDS in a private VPC subnet, separate from the web tier
Scalability	Vertical scaling only; resizing needs downtime	Web tier and DB storage scale independently
Availability	Single point of failure, no automatic recovery	Automated snapshots and optional Multi-AZ standby
Security	Credentials pointed at the local database on the same server	Secrets Manager plus security-group-only access
Maintenance	Manual backups, patching and disk planning	AWS manages backups, patching and storage

Technology Stack

Amazon EC2

Hosts CafeServer, the Apache/PHP web front end that serves the Café storefront.

Amazon RDS

Managed MariaDB (CafeDatabase) with automated backups, patching and storage management.

Secrets Manager

Stores /cafe/dbUrl, dbUser and dbPassword so the app reads credentials at runtime.

VPC & Subnets

Private subnets and security groups (CafeSG, dbSG) isolate and control traffic.

Systems Manager

Session Manager gives terminal access to CafeServer with no open SSH port.

SERVICE ROLES

Which Service Does What

Each AWS service had one clear job. Together they separate compute, data, secrets and network control.

Service	Role in this solution	Where it was used
Amazon EC2	Runs CafeServer: Apache and PHP serving the storefront; also the client used to run mysqldump and mysql	Steps 1, 2, 5, 6
Amazon RDS	Hosts the managed MariaDB database (CafeDatabase) that now stores all orders	Steps 3, 5, 6
AWS Secrets Manager	Holds /cafe/dbUrl, /cafe/dbUser, /cafe/dbPassword so the app finds the database without code changes	Steps 2, 6
VPC, Subnets, Security Groups	Place RDS in a private subnet and allow inbound traffic only from CafeSG (dbSG rule). Database port: 3306	Steps 3, 4
Systems Manager (Session Manager)	Gives browser-based terminal access to CafeServer with no open SSH port or key pairs	Steps 1, 2, 5, 6

IMPLEMENTATION STEPS

End-to-End Migration Workflow

01

Baseline & Access

Test the café app, place Order #25, connect via Session Manager.

02

Data Export

Run mysqldump to create CafeDbDump.sql from cafe_db.

03

RDS Creation

Launch CafeDatabase (MariaDB) in a private subnet, us-east-1a.

04

Security Groups

Allow dbSG inbound only from CafeSG.

05

Data Import

Load the dump into RDS and verify all 25 order rows.

06

Cutover & Proof

Update the secrets, stop local MariaDB, place Order #26.

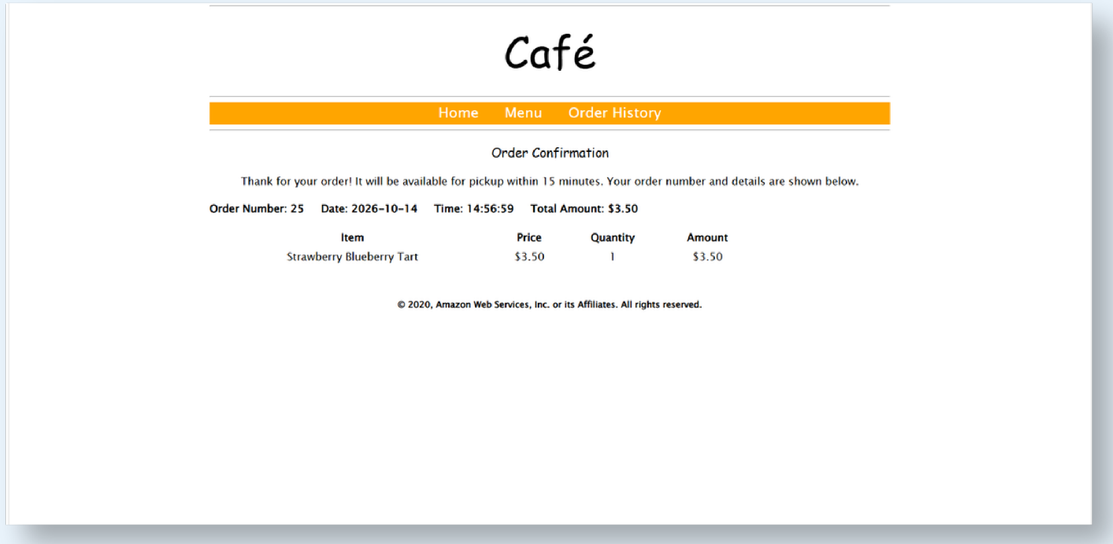
Each step is shown with its console evidence on the following slides.

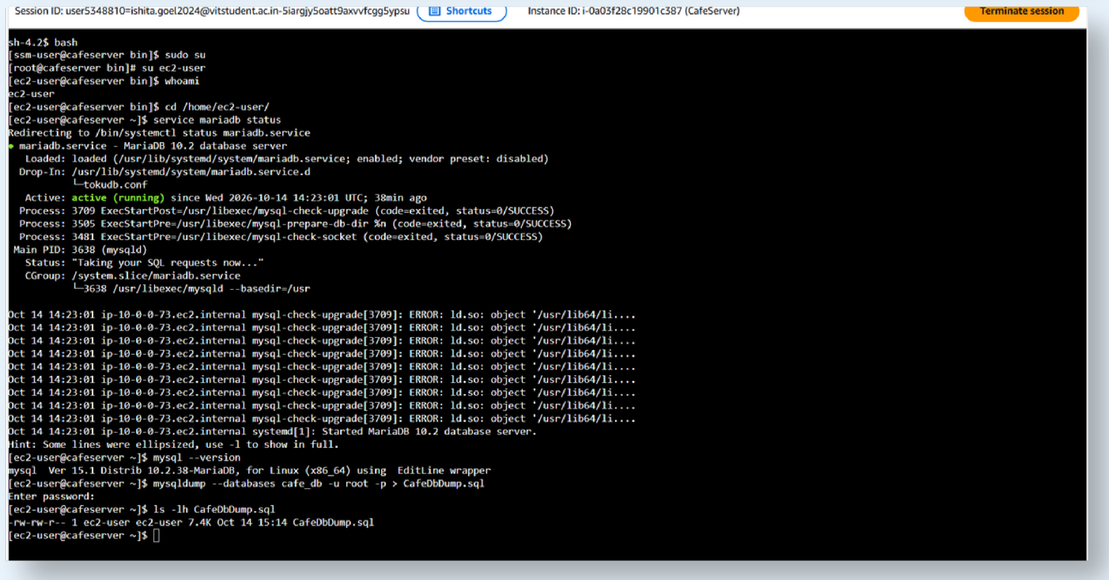
STEPS 1 & 2

Baseline Verification & Database Export

Before touching the database, we confirmed the app worked and then took a full backup of cafe_db so nothing could be lost.

- Opened the café site and placed Order #25 as a baseline
- Connected to CafeServer through Session Manager (no SSH)
- Retrieved root credentials from Secrets Manager
- Ran: mysqldump --databases cafe_db -u root -p > CafeDbDump.sql
- Verified the 7.4 KB dump with ls -lh



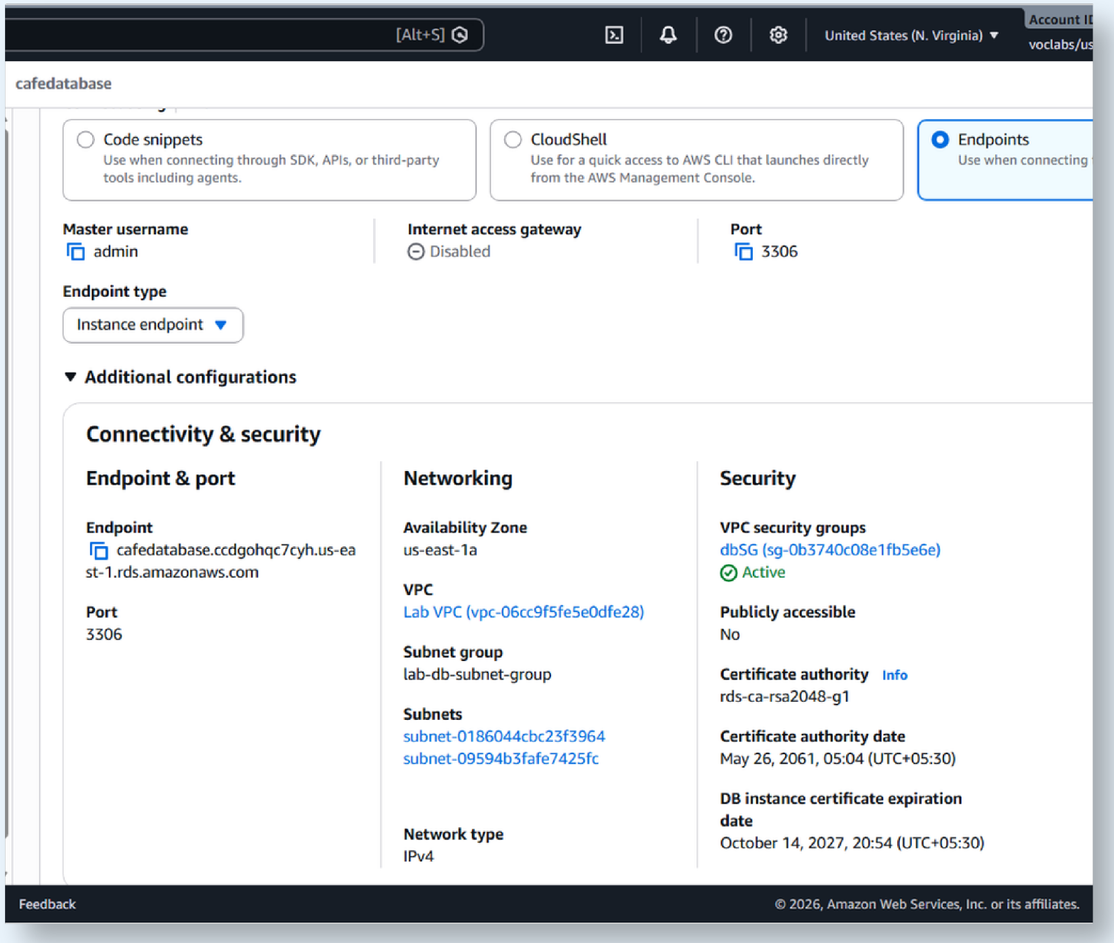


STEP 3

Amazon RDS Provisioning

We created a managed MariaDB instance so the café no longer runs or patches its own database server. A private subnet has no route from the internet, which is why a database belongs there.

- Instance identifier: CafeDatabase (MariaDB, master user admin)
- Placed in Lab VPC, subnet group lab-db-subnet-group
- Availability Zone: us-east-1a
- Public access: No; attached security group dbSG
- Database port: 3306 (default)
- Endpoint recorded for the later import and secret update

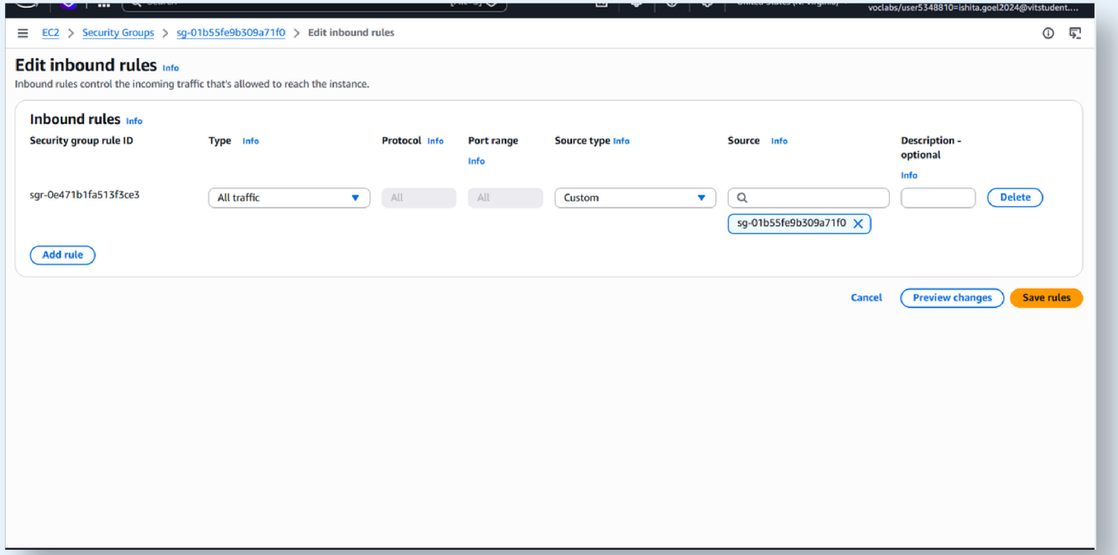


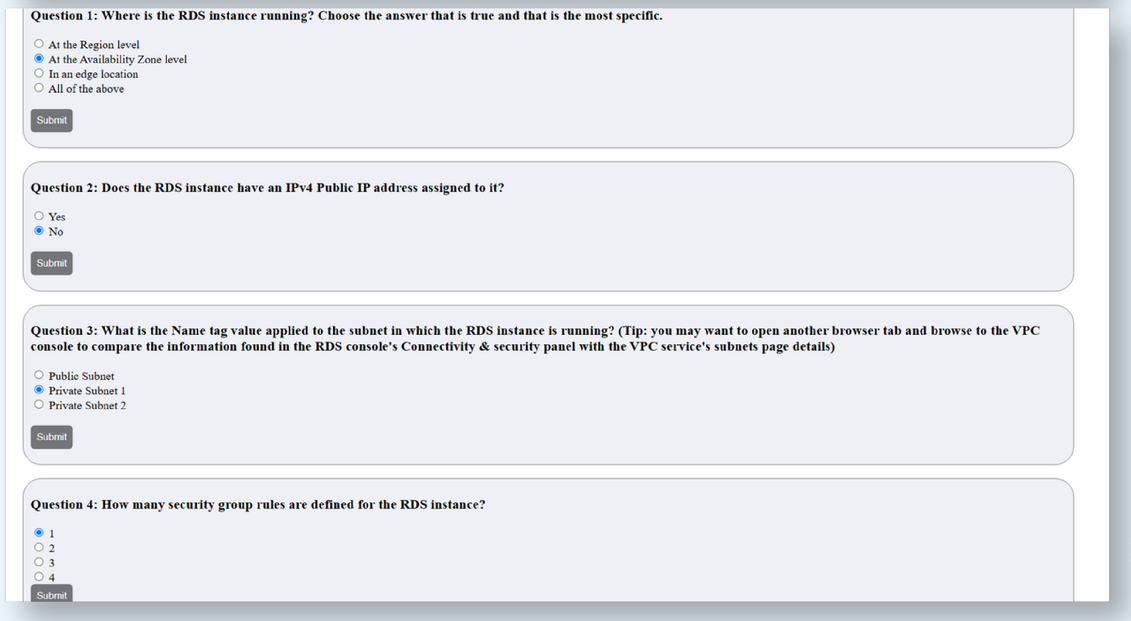
STEP 4

Network Hardening with Security Groups

Security groups are stateful virtual firewalls that only allow traffic. Naming CafeSG as the source means any instance in CafeSG can connect, and nothing else can, without tracking IP addresses.

- Edited inbound rules on dbSG
- Type = All traffic
- Source = CafeSG (sg-01b55fe9b309a71f0)
- No 0.0.0.0/0 rule, so no public exposure
- Knowledge checks confirmed AZ level, no public IP, Private Subnet 1 and 1 rule



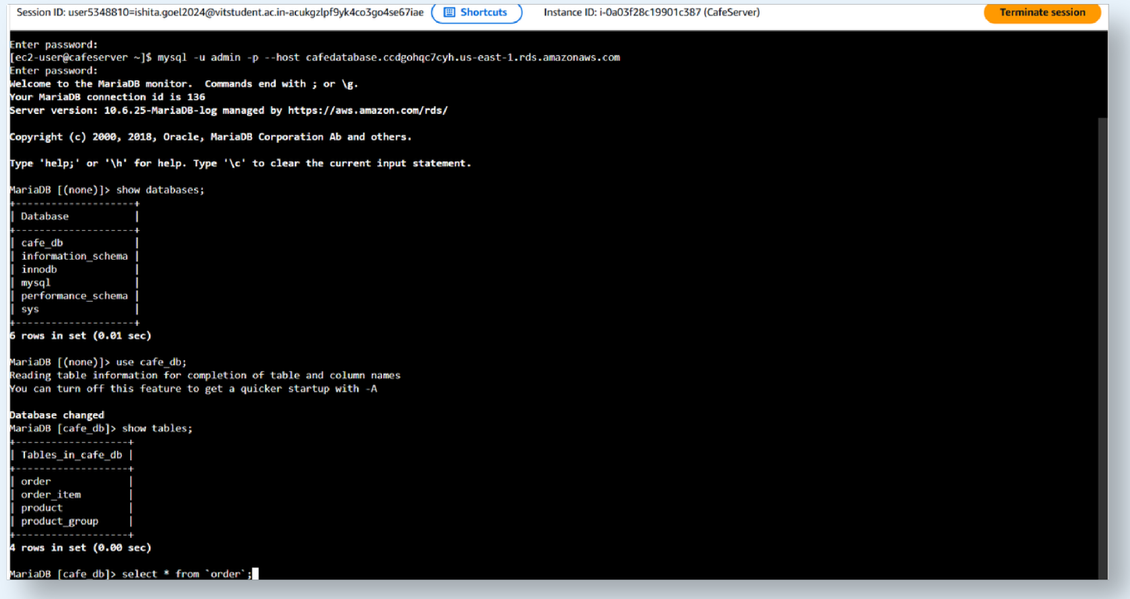


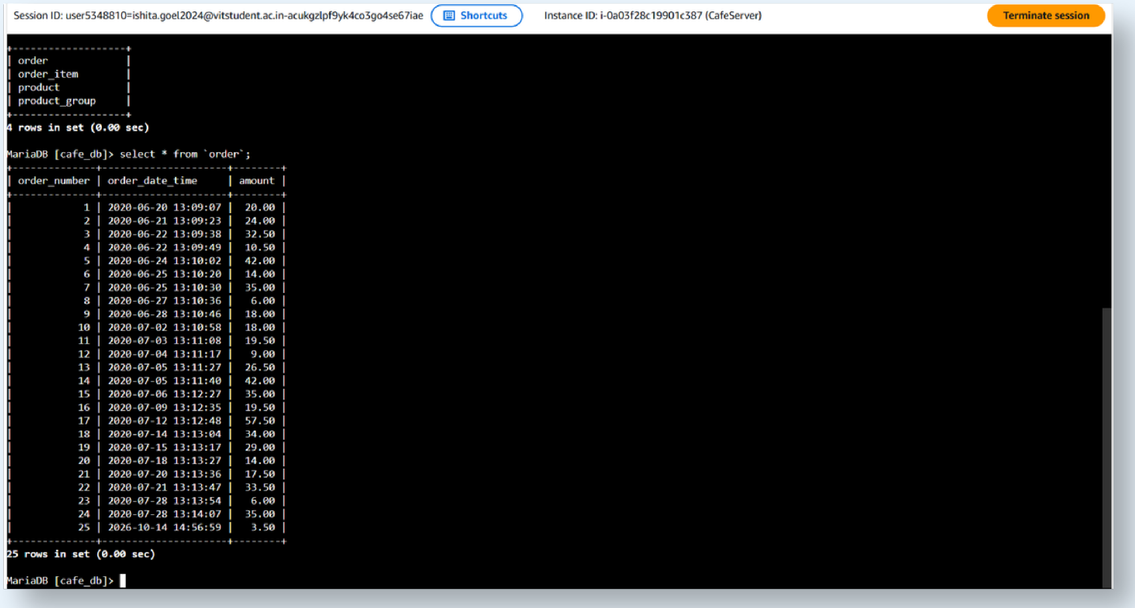
STEP 5

Data Import & Integrity Check

The dump was restored into RDS from CafeServer, then row counts were compared to prove no data was lost.

- Connected: mysql -u admin -p --host <RDS endpoint>
- Imported CafeDbDump.sql into the RDS instance
- Queried the order table after import
- Result: 25 rows in set, matching the source database



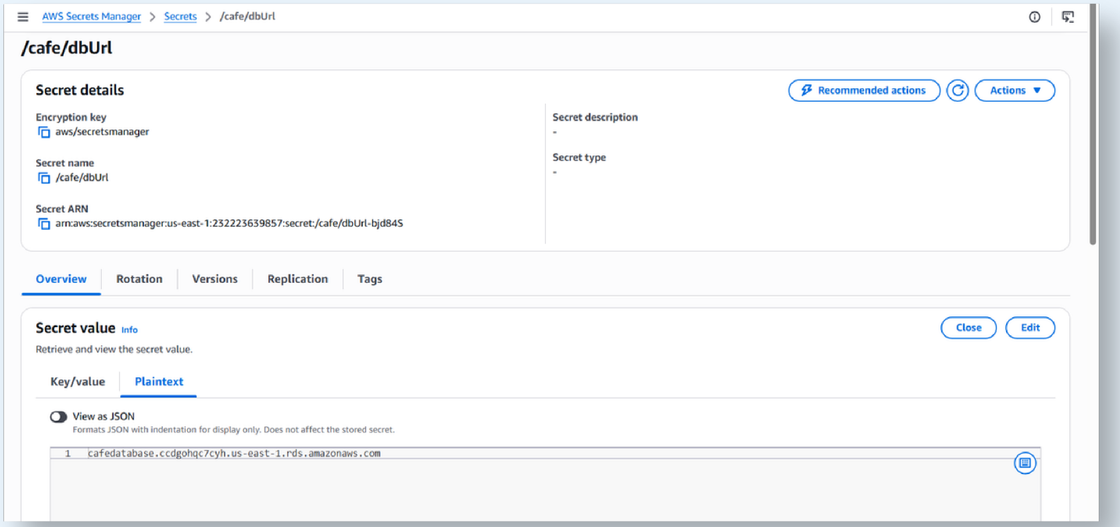


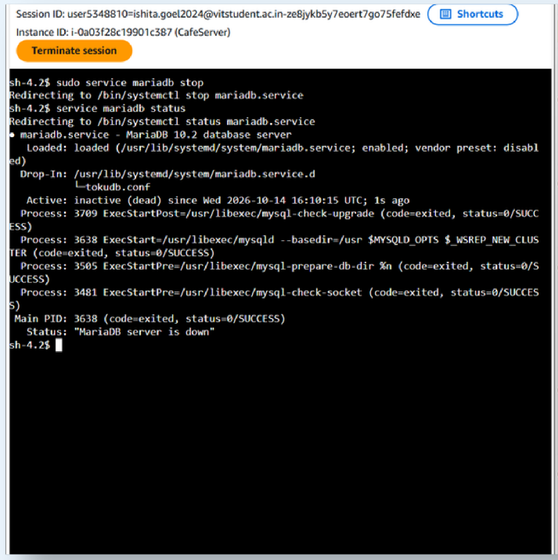
STEP 6

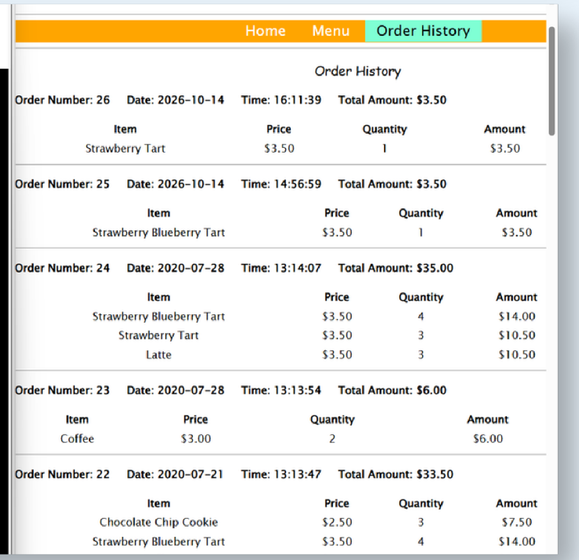
Secrets Manager Cutover & Decoupling Proof

The app reads its connection details from Secrets Manager, so we changed the secrets instead of the code.

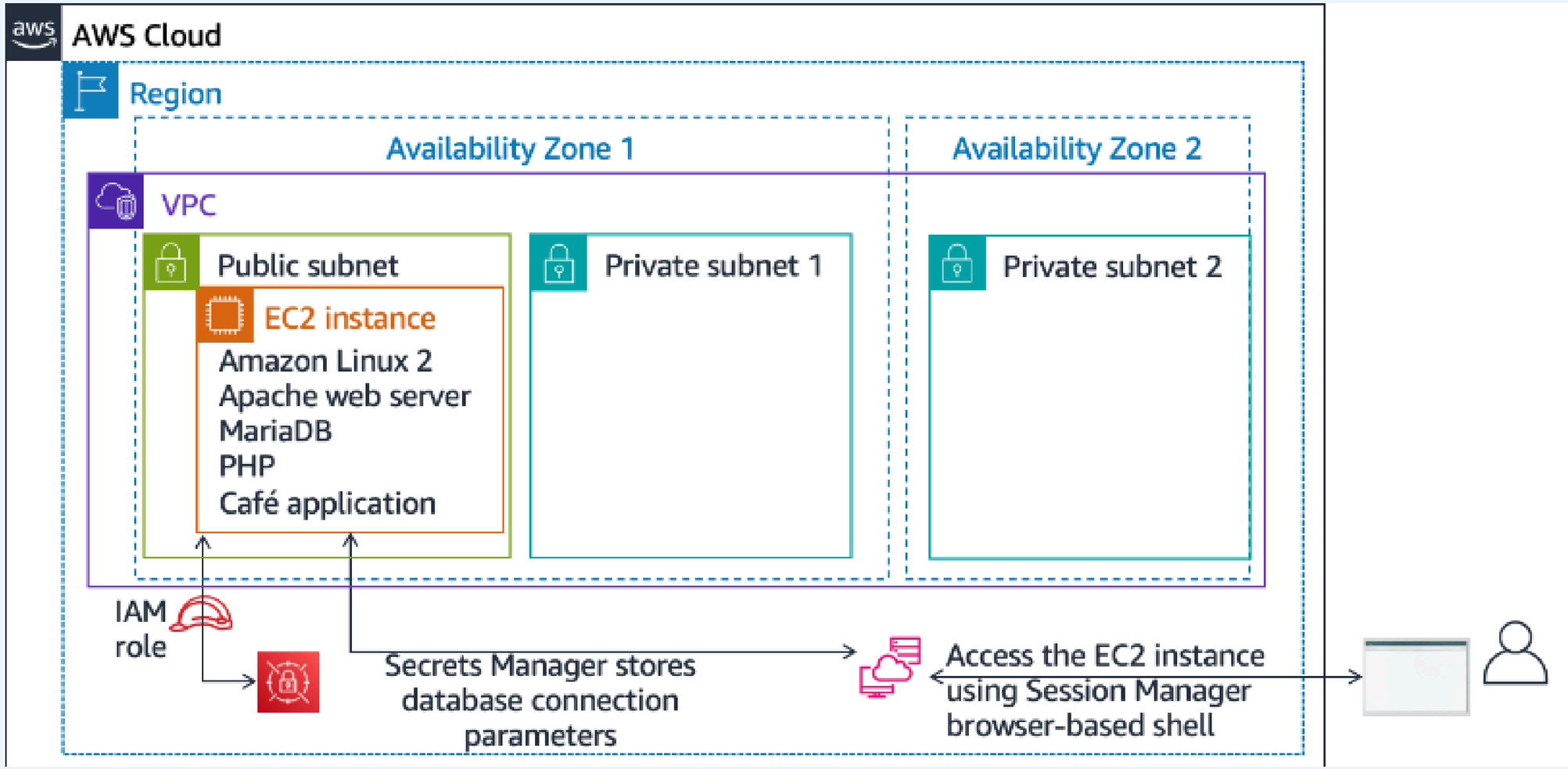
- Updated /cafe/dbUrl to the RDS endpoint
- Updated /cafe/dbUser and /cafe/dbPassword
- Stopped local MariaDB: sudo service mariadb stop
- Status showed inactive (dead)
- Placed Order #26 and saw it in Order History, served by RDS



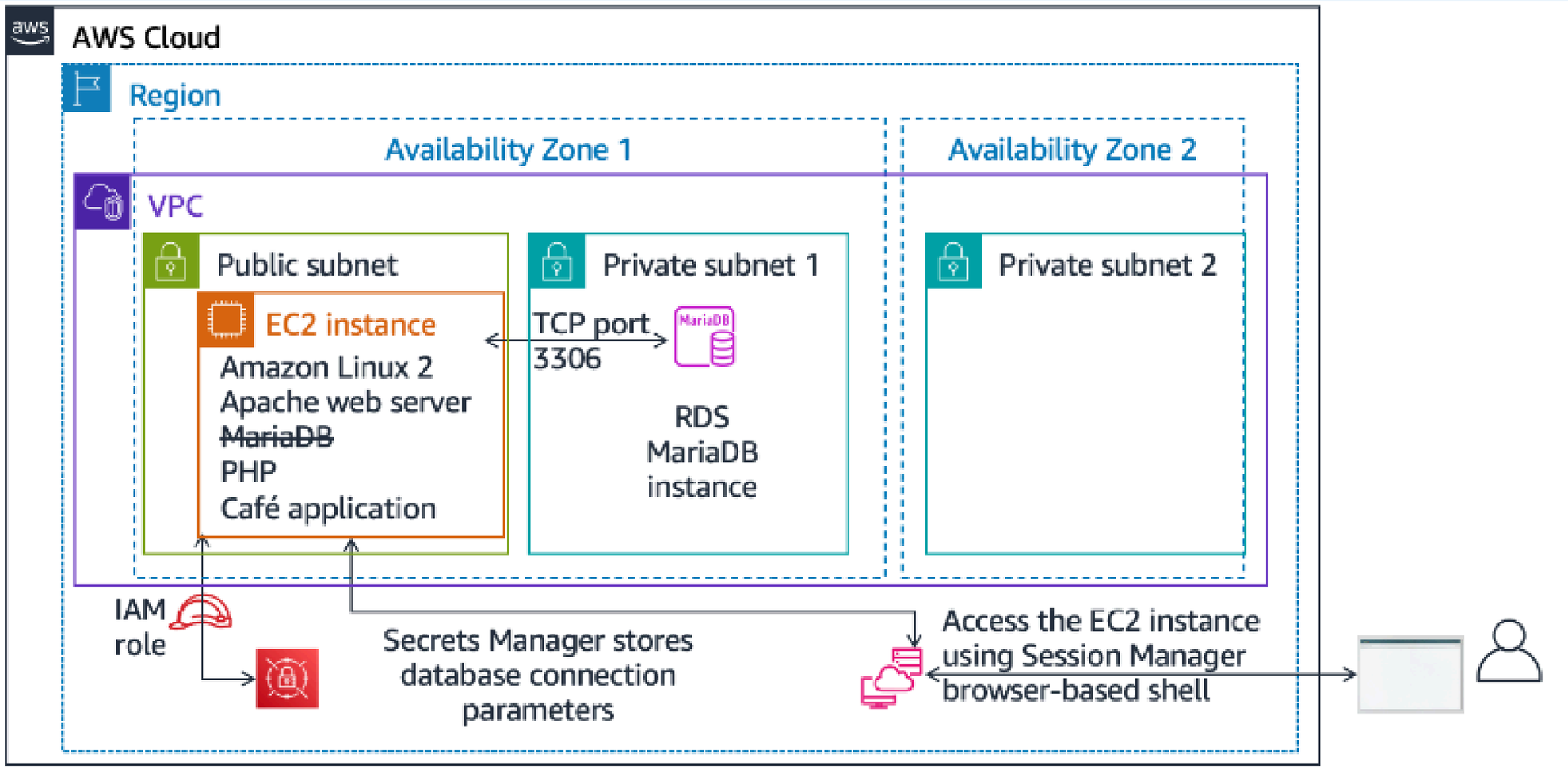




Pre-Migration: Initial Monolithic Architecture



Post-Migration: Target Decoupled Architecture

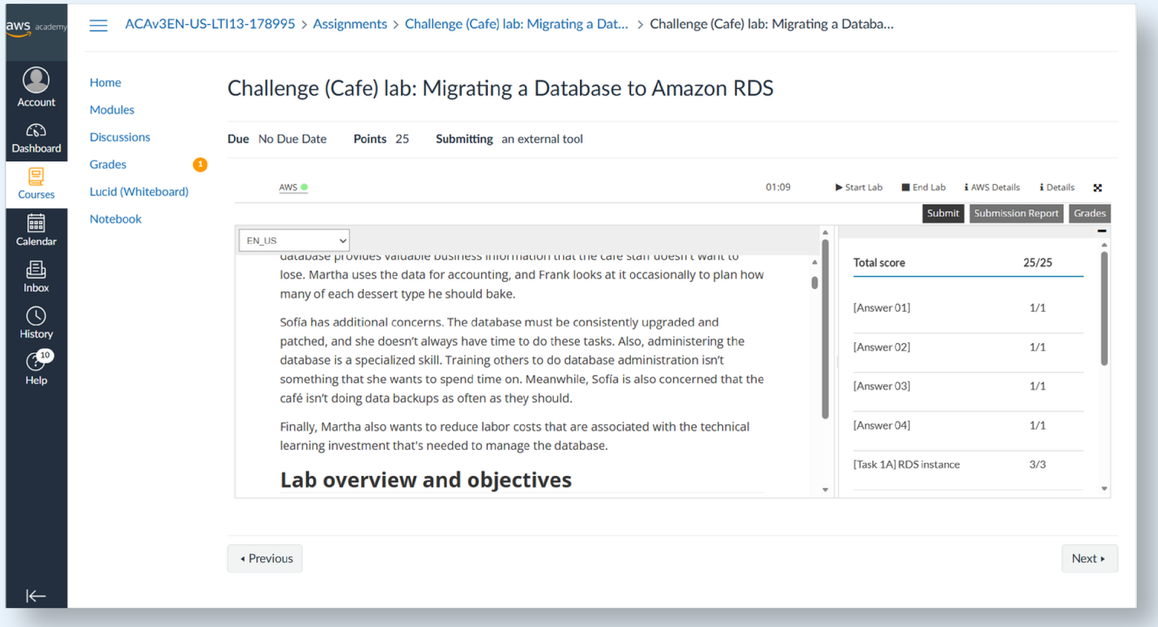


Challenges Addressed During the Migration

Challenge	How it was handled	Lesson
Needed root access to export data from a locally running database	Retrieved credentials from Secrets Manager, then ran mysqldump from CafeServer	Keep credentials out of scripts
RDS must be reachable by the app but never by the internet	Private subnet, Publicly accessible = No, dbSG allows only CafeSG	Use SG references, not IP ranges
Proving no data was lost	Compared the order table: 25 rows in RDS matched the source	Always verify after import
Switching databases without editing PHP code	Changed only the three /cafe secrets to point at RDS	Externalize configuration
Proving the app really uses RDS	Stopped local MariaDB, then placed Order #26 successfully	Test by removing the old dependency

VERIFICATION

Deployment Validation & Results



100%

Data preserved (25 of 25 rows)

0

Public database ports open

- Order #25 (before) and Order #26 (after) both appear in Order History
- Local MariaDB stopped, yet orders still work, so RDS is serving all traffic
- Automated evaluation confirmed every task: RDS instance, security group and secrets

Conclusion & Key Takeaways

What we achieved

- Web and database tiers decoupled and able to scale independently
- Database is private, reachable only from CafeSG
- Credentials stored in Secrets Manager, not in code
- Backups, patching and storage handled by AWS
- All 25 orders migrated with zero loss and the lab graded 100%

Key takeaways

- Managed services trade admin effort for configuration choices
- Security groups plus private subnets give defense in depth
- Externalizing config makes cutover a settings change
- Verify every migration step with evidence

AWS ACADEMY LAB 6 SEMINAR PRESENTATION

Thank You